

Questo progetto consiste nell'implementare una **versione semplificata di Pac-Man in C++17**, con due obiettivi principali:

1. **Rappresentare lo stato del gioco** come una griglia 2D serializzata tramite una struttura a **quadtree**.
2. **Gestire l'evoluzione del gioco**, mantenendo la cronologia completa di tutti gli stati (snapshot) generati durante la partita.

Devi implementare **due classi**:

`game_state`

Rappresenta **una singola fotografia (snapshot)** della partita:

- griglia di gioco;
- punteggio (score);
- vite (lives);
- numero di pellet rimasti;
- timer della modalità paura (*panic mode*).

`pacman`

Rappresenta l'intero gioco:

- contiene una lista concatenata di `game_state`;
- conserva tutta la cronologia degli stati;
- ogni mossa genera un nuovo `game_state` che viene aggiunto in coda.

1. Classe `game_state`

Griglia

La mappa è una matrice quadrata:

dove `size` può essere:

- 0
- oppure una potenza di 2 ≥ 4

Esempi:

La griglia contiene celle di tipo:

Tipo	Significato
empty	vuoto
wall	muro
pellet	pallina normale
power_pellet	super pallina
pacman	Pac-Man
ghost1..ghost4	fantasmi

Quadtree

La griglia NON viene salvata direttamente come matrice nell'XML.

Viene serializzata come **quadtree**:

- nodo foglia → regione uniforme
- nodo interno → diviso in 4 quadranti

Ordine dei quadranti:

1. alto-sinistra
2. alto-destra
3. basso-sinistra
4. basso-destra

Operazioni richieste

Costruttori

- default
- costruttore con dimensione
- copy constructor
- move constructor
- distruttore

Operatori

- copy assignment
- move assignment

Getter e setter

Per:

- size
- score
- lives
- pellets_left
- panic_countdown

Quadtree

La griglia NON viene salvata direttamente come matrice nell'XML.

Viene serializzata come **quadtree**:

- nodo foglia → regione uniforme
- nodo interno → diviso in 4 quadranti

Ordine dei quadranti:

1. alto-sinistra
2. alto-destra
3. basso-sinistra
4. basso-destra

Operazioni richieste

Costruttori

- default
- costruttore con dimensione
- copy constructor
- move constructor
- distruttore

Operatori

- copy assignment
- move assignment

Getter e setter

Per:

- size
- score
- lives

- pellets_left
- panic_countdown

Accesso celle

operator()(i,j) sia in versione const che non const.

Confronto

operator==

operator!=

Due stati sono uguali se:

- stessa dimensione
- stessi contatori
- stesse celle

Input/Output XML

Devi implementare:

operator<<

operator>>

Output

Scrivi il game_state nel formato XML richiesto.

Input

Legge XML e costruisce lo stato.

Deve lanciare: pacman_exception.

2. Classe pacman

Questa è la classe più importante.

Contiene: game_state -> game_state -> game_state -> ...

mediante lista concatenata.

Ogni nodo contiene: game_state gs; node* next;

Funzioni richieste

Gestione cronologia: push_back(game_state const&)

Aggiunge uno stato in fondo.

Controlla anche che:

- le dimensioni siano corrette;
- gli angoli dei fantasmi non contengano muri;
- il centro non contenga muri.

Movimento: `move(cell_type who, int di, int dj)`

È il cuore del progetto.

Prende l'ultimo stato della cronologia:

e genera un nuovo stato applicando una mossa.

Poi lo aggiunge alla lista.

Regole del movimento

Sono ammesse solo 4 direzioni:

`(-1,0)` // su

`(1,0)` // giù

`(0,-1)` // sinistra

`(0,1)` // destra

Qualunque altro valore → eccezione.

Muri e bordi

Se un personaggio prova a entrare:

- in un muro
- fuori dalla mappa

non si muove.

Tuttavia, il nuovo stato viene comunque salvato nella cronologia.

Collisioni Pac-Man/Fantasma

Modalità normale

Quando: `panic_countdown == 0`

e Pac-Man tocca un fantasma:

- perde una vita;
- tutti i personaggi vivi tornano alla posizione iniziale.
-

Pac-Man: (size/2, size/2)

Fantasm: ghost1 -> (0,0)

ghost2 -> (0,size-1)

ghost3 -> (size-1,0)

ghost4 -> (size-1,size-1)

ic Mode

Quando: panic_countdown > 0

Pac-Man mangia il fantasma.

Il fantasma:

- viene eliminato definitivamente;
- scompare dalla mappa.

Pac-Man guadagna punti.

Pellet

Pellet normale

Quando Pac-Man lo mangia: score += 10, pellets_left--

Power Pellet

Quando Pac-Man lo mangia: score += 50 pellets_left--

e attiva: panic mode

Sistema di punteggio

Evento	Punti
Pellet	10
Power Pellet	50
1° fantasma	200
2° fantasma	400
3° fantasma	800
4° fantasma	1600

Il bonus raddoppia ogni volta.

Panic Mode

Quando viene mangiata una power pellet: `panic_countdown = PANIC_RESET`

Ad ogni mossa: `panic_countdown--`

Quando arriva a 0: modalità normale.

Iteratori

La classe `pacman` deve implementare: `iterator` `const_iterator`

con: `begin()` `end()`

per scorrere la cronologia degli stati.

Serializzazione dell'intero gioco

Formato: XML

```
<pacman size="8">
```

```
  <game_state ...>
```

```
    ...
```

```
  </game_state>
```

```
  <game_state ...>
```

```
    ...
```

```
  </game_state>
```

```
</pacman>
```

Ogni snapshot viene scritto in ordine cronologico.

Cosa devi consegnare

Un solo file: `pacman.cpp`

Non devi modificare: `pacman.hpp`

In pratica

Questo progetto è soprattutto un esercizio su:

- gestione dinamica della memoria;
- Rule of Five (copy/move constructor e assignment);
- liste concatenate;
- iteratori personalizzati;
- parsing XML;
- serializzazione/deserializzazione;
- simulazione di logica di gioco;
- gestione di strutture dati gerarchiche (quadtree).

La parte più complessa è quasi certamente **implementare correttamente `pacman::move()`**, perché deve gestire tutte le regole di collisione, reset, punteggio e panic mode.